

Toward SFNNv16-Compatible Evaluation: Shared Bitboards, Incremental State, and a Provenance-Safe Eval-Bar Link

Manus AI
Unchessed UCI Engine Research
September 2026

Abstract—This paper reports a branch-local evaluator integration inspired by the public architectural principles of Stockfish 19’s SFNNv16. The implementation does not copy unavailable proprietary weights or claim exact SFNNv16 strength. Instead, it reuses the engine’s existing incremental NNUE accumulator seam, adds a zero-copy bitboard snapshot, routes the on-demand eval bar through the currently selected evaluator, and exposes a read-only link carrying Elo-detector telemetry. The design targets the high-value properties documented by Stockfish: sparse feature updates, accumulator reuse, quantized/integer-friendly inference, and training-aware provenance. On the real seven-position Matetrack corpus, the tensorized WDL conversion measured a 3.03x speedup over the exact logistic reference with zero per-mille output difference. The complete core suite passed 144 tests, and the UCI smoke test emitted the linked score, bitboard hash, material index, Elo estimate, confidence, and detector verdict without adding work to the search-node hot path.

I. SCOPE AND REPRODUCIBILITY

Stockfish 19 publicly reports that SFNNv16 removes redundant threat features, adds pawn-pair features, retires the secondary network used in an earlier release, and applies Quantization-Aware Training to very large rescored datasets [1]. The official NNUE documentation describes sparse features, feature-transformer accumulators, multiple perspectives, incremental updates, clipped activations, and quantized inference [2]. These public principles are reproducible. The exact SFNNv16 weights and full training pipeline are not present in this project, so the implementation is an independent compatible prototype rather than reverse-engineered weights.

All changes were made only on ‘manus/research-facilities’. The public implementation seams are ‘unchessed-core/src/eval_bar.rs’, ‘unchessed-core/src/uci.rs’, and ‘docs/evalbar-elo-bitboard-link.md’. The previous Stockfish 19 evaluation research is retained in ‘docs/ieee-stockfish19-evalbar-research.pdf’.

II. ARCHITECTURE DERIVED FROM PUBLIC SFNNv16 PRINCIPLES

A. Existing incremental evaluator seam

The engine already maintains one accumulator per perspective and threads ‘EvalState’ through search. A real

move updates only the changed feature rows; null moves reuse the parent state, and king-relative feature schemes rebuild only when the relevant king moves. The new bar does not create a second evaluator. ‘sample_with_state’ consumes an already available evaluator state and calls ‘eval_with_state’, preserving the accumulator’s performance contract.

B. Shared bitboard snapshot

The connection layer computes a compact immutable snapshot containing the position hash, total occupancy, White occupancy, Black occupancy, and the Stockfish-style material index. These values are copied from existing bitboards; no board scan, heap allocation, lock, or feature rebuild is required. The snapshot is carried in ‘EvalBarSample’ and ‘EvalBarLink’, providing a stable join key for UI, telemetry, and Elo-analysis consumers.

C. Evaluator routing

The UCI ‘evalbar’ command routes through the current evaluator object. The default hand-crafted evaluator is labeled ‘hce-proxy’; a loaded project NNUE is labeled ‘nnue-proxy’. A future exact SFNNv16-compatible weight file can use the same route without changing the connection contract. The source label prevents a proxy score from being mistaken for an official Stockfish score.

D. Fast WDL projection

The bar uses Stockfish’s public material-dependent calibration and WDL semantics [3]. For the normal display domain $[-4000, 4000]$, a one-time tensor stores rounded (W, D, L) per mille values for each material class and centipawn value. Runtime conversion is an integer offset and array access. Scores outside the domain use the exact fallback, and mate sentinels remain discrete.

III. CONNECTION INTERFACE

The stable Rust interface is:

```
let sample = bar.sample_with_state(
    &position, &evaluator, &state,
    EvalBarSource::LoadedNnueProxy);
let link = EvalBarLink::from_sample(
    &sample, elo.estimate_elo, elo.confidence_cp,
```

TABLE I
ARCHITECTURE COMPONENTS AND BOTTLENECK CONTROLS

Component	Reused or added capability	Bottleneck control
NNUE state	Existing per-perspective incremental accumulator	No duplicate feature rebuild
Bitboard link	Hash, occupancy, material, and side data	Copy-only, fixed-size snapshot
Elo link	Estimate, confidence, suspect, reason	Read-only telemetry snapshot
Evaluator route	HCE or loaded NNUE source label	On-demand UCI call only
WDL tensor	Cached material/cp lookup	No runtime exponential in normal range

TABLE II
FINAL MEASURED RESULTS

Metric	Result
Core tests	144 passed, 0 failed, 6 ignored
Eval-bar tests	8 passed, 0 failed
EPD positions	7
Rounds	10,000
Exact reference	4,974,389 ns
Fast tensor path	1,643,798 ns
Speedup	3.03x
Maximum WDL delta	0 per mille
Checksums	62,020,000 / 62,020,000

```
elo.suspect, elo.suspect_reason.name());
```

The UCI equivalent is ‘position ...’ followed by ‘evalbar’. The engine emits a score line and a machine-readable link line containing ‘hash’, ‘occ’, ‘material’, ‘elo’, confidence, ‘suspect’, and ‘reason’. The interface is documented in ‘docs/evalbar-elo-bitboard-link.md’.

The connection is deliberately one-way and read-only. It cannot change the Elo model, move selection, search limits, NNUE accumulator, or random state. This separation is important: a display feature must not become a hidden playing-strength feature.

IV. TESTING AND PERFORMANCE

The complete core library passed 144 tests with zero failures and six ignored tests. The eval-bar-specific suite passed eight tests. These tests cover bitboard-key equality, side-to-move normalization, rule-50 damping, WDL sum conservation, fast-versus-exact WDL agreement, symmetry, monotonicity, bounded smoothing, and the incremental-state seam.

The performance benchmark uses the repository’s seven-position legal Matetrack EPD corpus. It runs 10,000 rounds after tensor warm-up. The exact method evaluates the public logistic equations with ‘exp’; the fast method loads the precomputed tensor.

The final UCI smoke test returned ‘cp 14’, ‘wdl 32 957 11’, ‘bar 0.510’, and ‘source=hce-proxy’ for the start position. Its link line returned position hash ‘1780828403df6413’,

occupancy ‘ffff00000000ffff’, material index 78, Elo estimate 1500, confidence ± 450 , ‘suspect=false’, and ‘reason=none’.

V. LIMITATIONS AND NEXT STEPS

The measured 3.03x speedup applies only to WDL projection, not whole-engine nodes per second. The UCI command is intentionally on-demand. Search integration should use ‘sample_with_state’ at an existing evaluator boundary rather than calling the HCE convenience method at every node. Exact SFNNv16 quality cannot be established without the matching trained weights, feature definition, QAT procedure, and large rescored training corpus. A higher-quality independent version would therefore require a licensed or independently trained network, a provenance-matched SF19 output corpus, and SPRT/MAE/Brier calibration experiments.

VI. CONCLUSION

The implementation captures the most valuable public SFNNv16 ideas without pretending to reproduce unavailable artifacts: sparse incremental state, accumulator reuse, integer-friendly inference, and architecture-aware performance discipline. It also provides the requested eval-bar connection to Elo detection and bitboards through a stable read-only link. The result is a stronger and safer foundation for a future trained evaluator, while its proxy labels and benchmark scope keep the claims scientifically defensible.

REFERENCES

- [1] Stockfish Team, “Stockfish 19,” Sep. 5, 2026. [Online]. Available: <https://stockfishchess.org/blog/2026/stockfish-19/>
- [2] official-stockfish, “NNUE,” Stockfish Documentation. [Online]. Available: <https://official-stockfish.github.io/docs/nnue-pytorch-wiki/docs/nnue.html>
- [3] official-stockfish, “WDL model,” GitHub. [Online]. Available: https://github.com/official-stockfish/WDL_model
- [4] official-stockfish, “UCI Protocol and Stockfish Commands,” Stockfish Documentation. [Online]. Available: <https://official-stockfish.github.io/docs/stockfish-wiki/UCI-Protocol-and-Stockfish-Commands.html>